

Etapa 3 - Analizador Semántico (Chequeo de Declaraciones) de MiniJava

Compiladores e Intérpretes — UNS, 2026

Implementación del chequeo de declaraciones del analizador semántico de MiniJava. Se extiende el analizador sintáctico descendente recursivo de la Etapa 2 para que construya la Tabla de Símbolos (TS) a medida que reconoce las declaraciones, y se agrega una segunda pasada de control de declaraciones y la consolidación de clases e interfaces. No se chequean los cuerpos de métodos ni de constructores: esos controles corresponden a la Etapa 4.

1 Objetivo y logros

- Construir y consolidar la TS global de tipos, con las clases e interfaces declaradas y las entidades predefinidas.
- Controlar la correctitud de todas las declaraciones (nombres repetidos, tipos válidos, relaciones de herencia, circularidad, redefiniciones y contratos de interfaces).
- Reportar el primer error con un mensaje descriptivo y el código `[Error:lexema|nroLinea]`, o `[SinErrores]` si el programa es válido.

Se implementan los logros de la etapa anterior que siguen vigentes (atributos inicializados, operadores posfijos y visibilidad mejorada) y el manejo de archivos eficiente. Como logro de esta etapa se implementa **Genericidad Avanzada E3**: se admiten y controlan tipos genéricos anidados (por ejemplo `Caja<Par<String>>`) en atributos, retornos, parámetros y cláusulas de herencia e implementación. Este mismo trabajo cubre el logro **Genericidad Avanzada E2** (requisito del anterior): se admiten tipos genéricos anidados y la *notación diamante*, que solo se acepta al instanciar una clase genérica (`new Caja<>()`); si el tipo instanciado no declara un parámetro de tipo, o si el diamante aparece fuera de una instanciación, se reporta el error. No se implementan los restantes logros opcionales de la Etapa 3 (modificadores `sealed/final`, herencia múltiple de interfaces ni métodos genéricos), que requieren extender el léxico y la gramática.

2 Esquema general

El módulo principal recibe el archivo fuente y lo asocia al manejador de archivos y este al léxico. Luego crea la `TablaDeSimbolos` (que ya carga `Object`, `String` y `System`) y el `AnalizadorSintactico` con dependencia al léxico y a la TS. El sintáctico, mientras valida la sintaxis, ejecuta las acciones semánticas que insertan las entidades en la TS y realizan los controles de la primera pasada. Al finalizar el parseo, el módulo principal ejecuta el chequeo de declaraciones (`estaBienDeclarada`) y la consolidación (`consolidar`). Si no hay errores informa el mensaje de éxito; ante un error léxico, sintáctico o semántico captura la excepción correspondiente, reporta el mensaje y el código de error, y finaliza la ejecución.

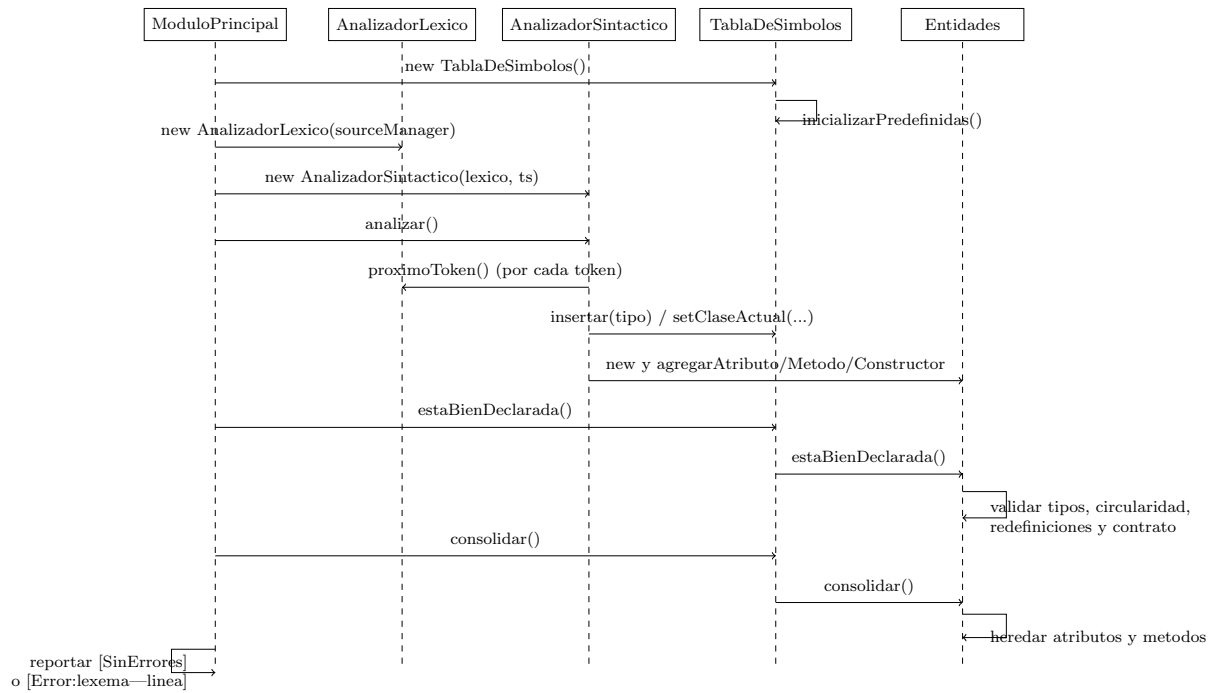


Figure 1: Diagrama de secuencia del análisis de la Etapa 3.

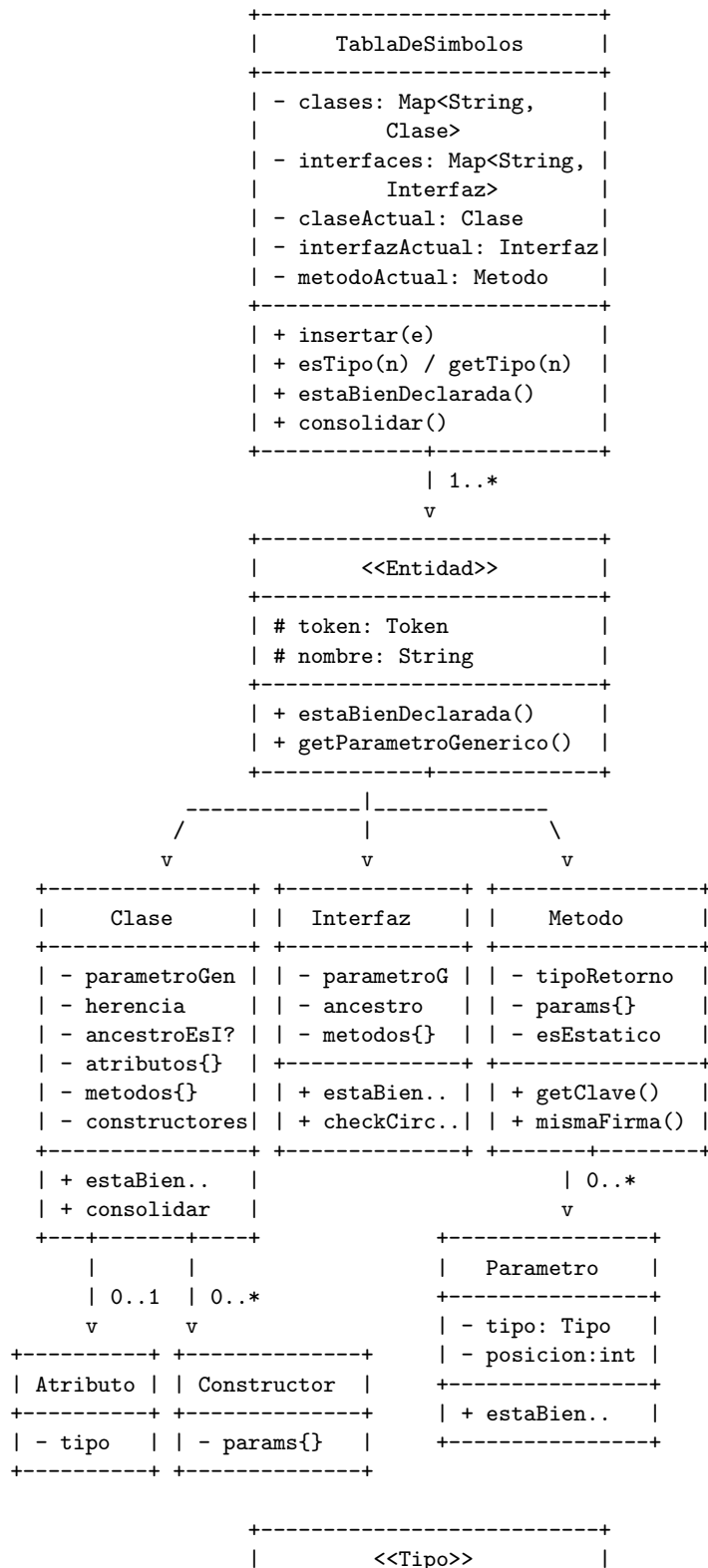
3 Estructura del proyecto

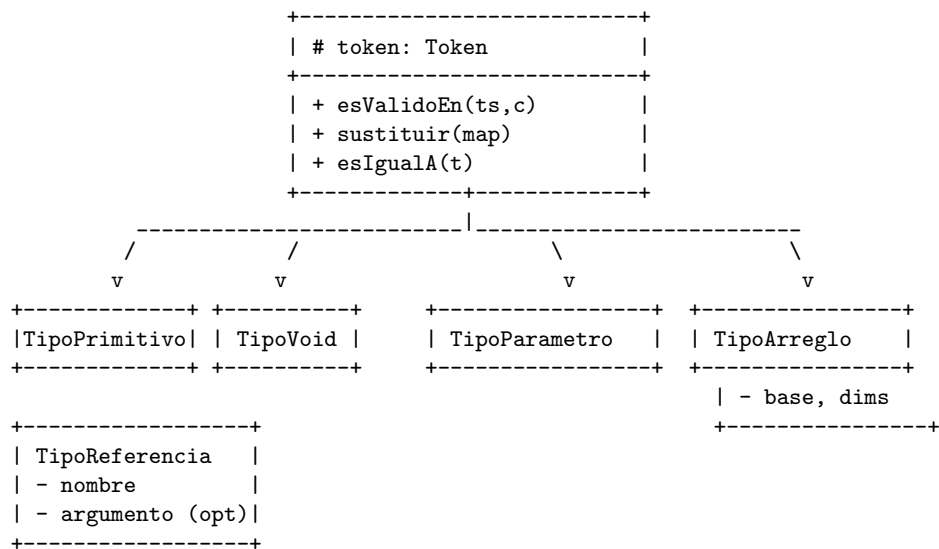
compilador-minijava/	
-- Compilar.sh / Compilar.bat	Compila y genera Compilador.jar
-- CorrerTests.sh / CorrerTests.bat	Compila y ejecuta los testers JUnit
-- lib/	Jars de JUnit 4 y Hamcrest (se autodescargan)
-- src/main/java/	
-- moduloprincipal/	
-- ModuloPrincipal.java	Modulo principal (interfaz con el usuario)
-- analizadorlexico/	Etap 1 (sin cambios)
-- analizadorsintactico/	
-- AnalizadorSintactico.java	Parser + acciones semanticas (construye la TS)
-- ExcepcionSintactica.java	Error sintactico
-- analizadorsemantico/	NUEVO: Tabla de Simbolos y chequeos
-- TablaDeSimbolos.java	Tabla global, predefinidas, puntos de entrada
-- Entidad.java	Entidad declarable (abstracta)
-- Clase.java	Clase concreta
-- Interfaz.java	Interfaz
-- Metodo.java	Metodo
-- Constructor.java	Constructor
-- Atributo.java	Atributo de instancia
-- Parametro.java	Parametro formal
-- Tipo.java	Tipo (abstracto)
-- TipoPrimitivo.java	boolean / char / int
-- TipoVoid.java	void (solo retorno)
-- TipoParametro.java	Parametro de tipo (T)
-- TipoArreglo.java	Arreglo con dimensiones
-- TipoReferencia.java	Clase/interfaz con argumento generico opcional
-- ExcepcionSemantica.java	Error semantico
-- sourcemanager/	Etap 1 (sin cambios)
-- src/test/java/test/	Testers JUnit provistos por la catedra
-- resources/	
-- sinErrores/	Casos de prueba sin errores
-- conErrores/	Casos de prueba con errores

4 Tabla de símbolos

Cada entidad declarable se representa con una clase que conserva el token de su declaración (lexema y número de línea) y sabe controlarse a sí misma. La tabla global comparte el espacio de nombres de clases e interfaces y mantiene las clases predefinidas.

4.1 Diagrama de clases





4.2 Responsabilidades

- **TablaDeSimbolos:** administra el espacio global de tipos, las entidades predefinidas y el contexto actual; recorre la tabla para el chequeo y la consolidación.
- **Clase/Interfaz:** controlan su relación de herencia, su circularidad, sus tipos y sus miembros; consolidan su tabla respectiva.
- **Metodo/Constructor/Atributo/Parametro:** controlan la validez de sus tipos y de sus nombres.
- **Tipo y sus subclases:** modelan de forma uniforme los tipos primitivos, void, parámetros de tipo, arreglos y referencias con argumento genérico; implementan validez, sustitución e igualdad estructural.

5 Chequeo de declaraciones

5.1 Primera pasada (construcción de la TS)

Se ejecuta durante el parseo, al insertar cada entidad. Controla:

- nombres de tipo repetidos (y redeclaración de **Object**, **String** y **System**, que ya están en la tabla);
- dos atributos con el mismo nombre en una clase;
- dos métodos con la misma clave (**nombre**, **aridad**) en una clase o interfaz, aunque difieran en los tipos o nombres de sus parámetros;
- dos constructores con la misma aridad en una clase;
- parámetros repetidos en una misma unidad (método o constructor);
- que el nombre de cada constructor coincida con el de su clase.

5.2 Segunda pasada (estaBienDeclarada)

Se ejecuta una vez finalizado el parseo. Controla:

- que todo tipo referencia usado en atributos, retornos y parámetros exista (clase, interfaz o entidad predefinida);
- que el argumento genérico sea válido: el tipo referenciado debe declarar un parámetro de tipo y el argumento debe ser un parámetro de tipo válido en el contexto o un tipo existente; el control es recursivo, de modo que admite y valida **tipos genéricos anidados** (por ejemplo `Caja<Par<String>>`, `Base<Par<Caja<T>>>`);
- que un parámetro de tipo solo se use dentro de la clase o interfaz que lo declara;
- que **extends** apunte a una clase existente, **implements** a una interfaz existente y la extensión de interfaz a otra interfaz;
- la ausencia de herencia circular entre clases y de extensión circular entre interfaces;
- la redefinición de métodos de instancia (mismo nombre y aridad que un método heredado exige signatures exactamente iguales: misma cantidad, mismos tipos de parámetros en orden y mismo retorno, tras instanciar los genéricos);
- los conflictos con métodos estáticos heredados (un método estático heredado con una clave prohíbe cualquier método con esa clave, estático o de instancia, y un método estático nuevo no puede chocar con un método de instancia heredado);
- el contrato de interfaces: la clase debe poseer, directa o heredadamente, un método de instancia compatible con cada método de la interfaz (un método estático no la implementa).

5.3 Consolidación

La consolidación incorpora los miembros heredados con delegación y recursión: una clase se consolida solo después de su ancestro directo. Al heredar de un tipo genérico, los tipos paramétricos de los miembros heredados se instancian con el argumento explícito (por ejemplo **extends** `A<String>`) o conservan el nombre del parámetro del tipo heredado si no hay argumento (por ejemplo **extends** `A`). Durante la consolidación se controla además el choque de un atributo propio con un atributo heredado. Las redefiniciones y el contrato de interfaces se controlan en la segunda pasada, antes de consolidar.

6 Entidades predefinidas

Antes de analizar el programa la tabla contiene:

- **Object**, que declara `String toString()`;
- **String**, tipo de los literales de cadena, que hereda de **Object** y no declara métodos propios;
- **System**, que hereda de **Object** y declara los métodos estáticos `debugPrint(int)`, `read()`, `printB(boolean)`, `printC(char)`, `printI(int)`, `printS(String)`, `println()`, `println(boolean)`, `println(char)`, `println(int)` y `println(String)`.

7 Reporte de errores

Al detectarse un error se reporta y finaliza la ejecución. El token asociado sigue las pautas de la cátedra: para un nombre repetido es el token declarado después; para un tipo no declarado, el token que lo referencia; para un método mal redefinido, el token de su nombre en la clase descendiente; y cuando no hay un token claramente asociado (por ejemplo, un método de interfaz no implementado), el token del nombre de la entidad con el problema.

```
Error Semantico en linea <N>: <mensaje>
[Error:<lexema>|<N>]
```

8 Formato de salida

8.1 Análisis exitoso

```
Compilacion Exitosa
[SinErrores]
```

8.2 Análisis con errores

Mensaje descriptivo seguido del código de error, por ejemplo:

```
Error Semantico en linea 7: el tipo A1 ya fue declarado
[Error:A1|7]
```

9 Compilación y ejecución

No se usa ninguna herramienta de build: solo se necesitan `javac`, `jar` y `java` de cualquier JDK 21 o superior (probado con JDK 25).

```
./Compilar.sh           # compila y genera Compilador.jar (Windows: Compilar.bat)
java -jar Compilador.jar resources/sinErrores/semCorrecto01.java
./CorrerTests.sh        # compila y ejecuta los testers JUnit (Windows: CorrerTests.bat)
```

9.1 Comandos manuales equivalentes

Compilar (desde `compilador-minijava/`):

```
javac -d build/classes \
  src/main/java/moduloprincipal/*.java \
  src/main/java/analizadorlexico/*.java \
  src/main/java/analizadorsintactico/*.java \
  src/main/java/analizadorsemantico/*.java \
  src/main/java/sourcemanager/*.java
jar cfe Compilador.jar moduloprincipal.ModuloPrincipal -C build/classes .
```

Correr los tests (desde `compilador-minijava/`, con JUnit 4.13.2 y Hamcrest 1.3 en `lib/`):

```
javac -cp "build/classes:lib/junit-4.13.2.jar:lib/hamcrest-core-1.3.jar" \
  -d build/test-classes src/test/java/test/*.java

java -cp "build/classes:build/test-classes:lib/junit-4.13.2.jar:\
lib/hamcrest-core-1.3.jar" org.junit.runner.JUnitCore \
  test.TesterDeCasosSinErrores test.TesterDeCasosConErrores
```

En Windows el separador del classpath es `;` en lugar de `:`.

10 Tests

Se incluyen los casos provistos por la cátedra y casos propios desarrollados para esta etapa (el diseño de los casos de prueba es responsabilidad del alumno). Resultado actual: **100/100 casos OK**.

Los casos sin errores cubren: predefinidas y redefinición válida de `toString`; herencia encaenada; redefinición exacta; sobrecarga por aridad; implementación de interfaces (simples, con extensión y genéricas); clases genéricas con parámetros en atributos, retornos y parámetros; herencia genérica con y sin argumento; redefinición tras instanciar genéricos; tipos genéricos anidados en atributos, `extends` e `implements`; notación diamante al instanciar clases genéricas (`new Caja<>()`); constructores de distinta aridad y constructor por defecto; arreglos multidimensionales; y métodos estáticos.

Los casos con errores cubren: nombres de tipo repetidos; redeclaración de las predefinidas; `extends/implements`/extensión a tipos inexistentes o de tipo equivocado; herencia y extensión circular; atributos repetidos y choque con atributos heredados; métodos repetidos por (`nombre, aridad`); redefiniciones con firma distinta (incluida la redefinición inválida de `toString` de `Object` en descendientes directos e indirectos, y en clases que implementan interfaces); conflictos entre métodos estáticos y de instancia heredados; métodos de interfaz no implementados o implementados de forma estática; constructores con aridad repetida o con nombre distinto al de la clase; parámetros repetidos; tipos inválidos en atributos, retornos y parámetros; argumentos genéricos sobre tipos no genéricos o con tipos inexistentes (incluidos los anidados); uso del diamante sobre una clase no genérica; y uso de parámetros de tipo fuera de su contexto.